RAG Assignment Report: Baseline vs. Re-Ranking

Hrigved Naidu (2026701025)

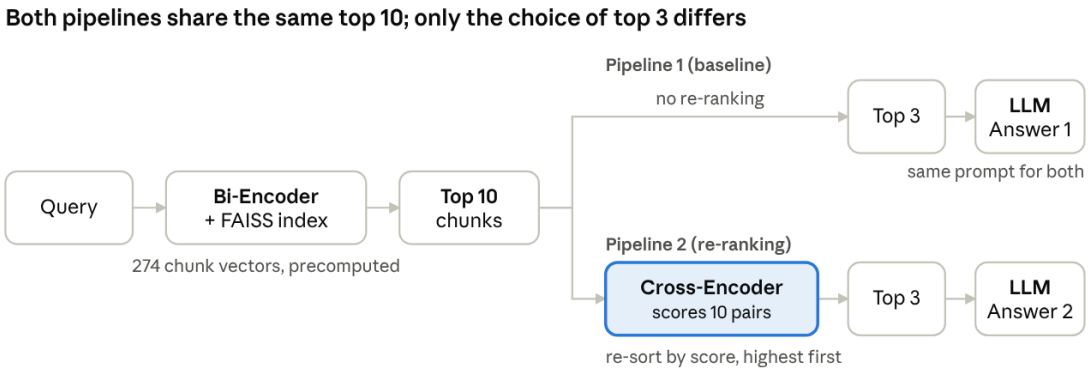
IIIT - Hyderabad

Setup

I compared a simple Bi-Encoder retriever to the same retriever followed by a Cross-Encoder re-ranker on one PDF, the ICC Men's T20 World Cup 2026 Playing Conditions (106 pages, about 46,600 words after removing the page headers and the table of contents). The code is in one notebook (link in the submission).

Component	Choice
Chunking	200-word windows, 30-word overlap, 274 chunks (longest is 482 tokens)
Bi-Encoder	BAAI/bge-small-en-v1.5, normalized vectors, FAISS inner-product index (= cosine)
Cross-Encoder	cross-encoder/ms-marco-MiniLM-L-6-v2
LLM	gemini-3.1-flash-lite, temperature 0, same prompt for both pipelines
Stress tests	25 queries: 22 written by me, 3 generated by the LLM from random chunks

I chose 200 words by counting the number of tokens in each chunk. The Cross-Encoder accepts a maximum of 512 tokens for the query plus chunk, so 300 words had 1.7% chunks that were too long, 400 words had 97%.



two pipelines · same top 10, different top 3

Both pipelines use the same 10 candidates and the same prompt. Only the way the 3 chunks sent to the LLM are chosen differs.

1. Architecture comparison

A Bi-Encoder encodes the query and each chunk separately into one vector each. Relevance is the cosine similarity of the two vectors. Since chunks are never presented with queries at inference time, a Bi-Encoder can precompute all chunk vectors and simply perform a nearest-neighbour search of the query vector at search time. The problem with this approach is that a chunk's meaning is compressed into one vector, losing nuance like negations, exceptions, or exact wording.

A Cross-Encoder takes the query and one chunk as a combined input to the transformer, and computes one attention matrix (self-attention) across the combined tokens. This allows every query word to attend to every chunk word, and the model outputs one relevance score for the pair. It requires much more computation than a Bi-Encoder, but produces much more accurate scores, and nothing can be precomputed since scores are specific to (query, chunk) pairs.

	Bi-Encoder	Cross-Encoder
Input	Query and chunk encoded separately	Query and chunk encoded together
Interaction	Only through cosine of two vectors	Token-to-token attention across both
Precompute chunks?	Yes, once	No
Output	A vector	A single relevance score
Role here	Find the top 10 from 274 chunks	Re-order those 10

2. Qualitative and rank-shift analysis

Out of 25 questions of stress, the gold chunk was already among the top 3 answers in the baseline for 14, moved to top 3 after re-ranking for 4, stayed at positions 4-10 in 2, and was not found in top 10 at all for 5. Consequently, gold in top 3 increased from 14/25 (56%) to 18/25 (72%), with a theoretical max of 20/25 (80%) since re-ranking can only consider the top 10 candidates.

Rank changes

Query	Category	Gold chunk	Baseline rank	Re-ranked	Result
Can a batter be bowled when the umpire has called a Wide?	Exception / negation	C109	#6	#2	Fixed
If a person filling in for an injured teammate breaks a serious rule, who is held responsible?	Other (LLM-generated)	C197	#10	#2	Fixed

Query	Category	Gold chunk	Baseline rank	Re-ranked	Result
What happens if a player is caught tampering with the ball a second time?	Other (LLM-generated)	C163	#6	#3	Rank only, answer did not improve
What happens on the ball after a No ball is bowled?	Exception	C105	#10	#5	Better, still outside top 3
Can a batter be bowled if the ball touched a fielder first?	Exception / negation	C142	#9	#4	Better, still outside top 3

In the Wide query, the baseline put chunk C107 (calling and signaling a Wide) first, as it matches the query words, but doesn't discuss possible dismissals. The chunk with the answer, C109 (clause 22.9, 'neither batter shall be out under any of the Playing Conditions except...') sat at #6, with a Cross-Encoder with score of 5.24, it put C109 at 2nd position.

Answer 1 vs Answer 2

Query	Answer 1 (baseline top 3)	Answer 2 (re-ranked top 3)	What happened
Bowled after a Wide?	"The provided context does not state whether a batter can be bowled after the umpire has called a Wide."	"No. A batter can only be out by hit wicket, obstructing the field, run out or stumped."	Baseline omitted the clause that answers the question
Substitute breaks a serious rule	"The context does not contain information regarding who is held responsible."	Cites clause 42.4.2: the nominated player who has a substitute also suffers the penalty for a Level 4 offence by the substitute.	Baseline omitted the clause

In both cases the LLM did not hallucinate. Because the critical chunk was outside the top 3, it correctly said the context was insufficient, but it still failed the user. The error is in retrieval, not generation.

Where re-ranking did not help

Five queries (bouncers per over, number of reviews per innings, LBW off a No ball, time to be ready for the next over, bowler switching to round the wicket) had the gold chunk outside the top 10, and so never seen by the Cross-Encoder. These are Bi-Encoder recall failures, and only better retrieval (different chunking, a stronger embedding model, hybrid keyword search) could fix them.

The review count query is another keyword trap example. Both pipelines retrieved the Super Over chunk, which shares the words 'review' and 'innings', and both answers stated the Super Over limit (one unsuccessful review) as if it were the general rule, a misstatement prompted by a distractor chunk.

For the Wide query, the re-ranking fixed an exception-and-negation case, but for the questions about being bowled after a fielder touched the ball, and about the ball after a No ball, improved the rank, but not enough to make it to the top 3.

3. Latency profiling

Re-ranking made the retrieval stage about 39 times slower (56 ms to 2,167 ms), measured on a Colab CPU runtime. Each figure is a mean over 5 different queries, after one warm-up run.

Stage	Mean time (ms)
Pipeline 1: retrieval (embed query + top-10 search)	55.7
Re-ranking step only (10 pairs)	2,111.8
Pipeline 2: retrieval + re-ranking	2,167.4
LLM generation (1 call, same for both)	1,350.7

The Cross-Encoder took about 211 ms per (query, chunk) pair on CPU. Since the LLM call takes the same amount of time in both pipelines, adding it to the end-to-end time makes the difference about 1.6x (1.4 s vs 3.5 s per query). These numbers are CPU-only and noisy, but using a GPU would reduce the re-ranking time significantly.

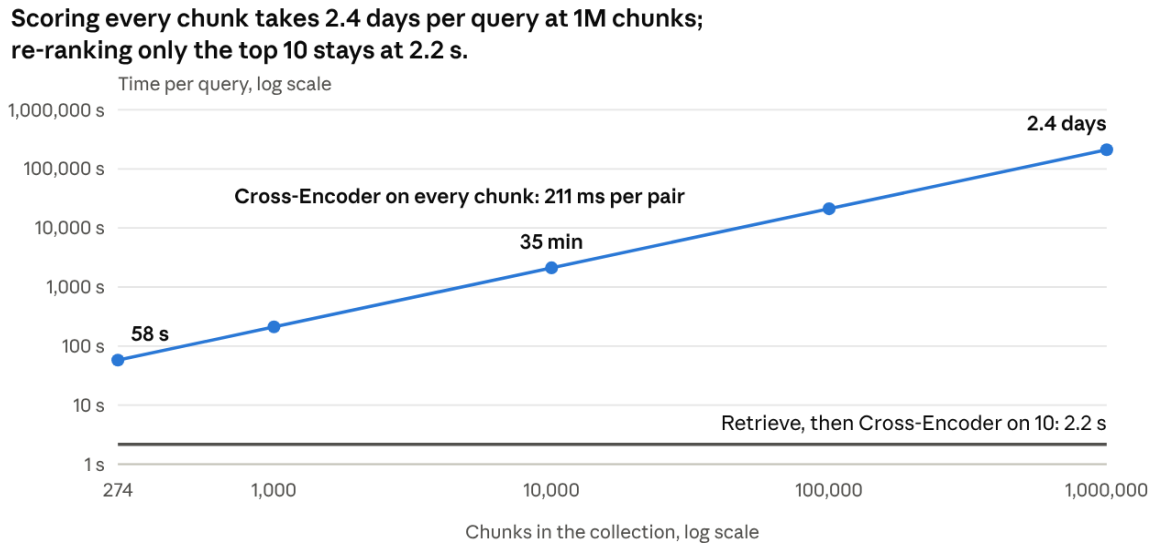
4. Scaling question

A Cross-Encoder can not be precompute anything, hence executing it on the entire collection would require a single transformer pass for every query and every chunk. This means linear scaling in terms of the corpus size and a cost that is re-paid with every query. A Bi-Encoder can encode every chunk once, offline, and only encodes the query at query time, and performs a vector search, which remains fast even for millions of chunks (and is even faster with approximate indices). That's why the Bi-Encoder narrows down the collection to 10 candidates, and only then the Cross-Encoder is applied on these candidates.

Using my measured 211 ms per pair on CPU:

Corpus size (chunks)	Cross-Encoder on every chunk, per query
274 (this document)	58 s
1,000	3.5 min
10,000	35 min
100,000	5.9 hours
1,000,000	2.4 days

By comparison, the two-stage pipeline stays at about 2.2 s per query, because the Cross-Encoder always sees only 10 pairs.



211 ms per pair measured on Colab CPU · larger sizes extrapolated linearly.